Reproducing the Temeraire Redis Results on WSL2 and Bare-Metal Linux

Aldo Costa
aldo.costa@hhu.de
Heinrich-Heine-Universität
Düsseldorf, Nordrhein-Westfalen, Deutschland

Abstract

Temeraire is a hugepage-aware extension of TCMalloc that aims to improve application performance by preserving hugepage coverage and reducing TLB pressure. The original OSDI 2021 paper evaluates Temeraire through application studies, a Google fleet experiment, and a production rollout. This report deliberately narrows reproduction to the part that can be approximated with public code: the Redis 6.0.9 case study.

The artifact began as a generic allocator benchmark. Turning it into a Redis reproduction meant pinning Redis 6.0.9, building legacy and hugepage-aware libraries from a historical public TCMalloc revision, and verifying the loaded allocator at runtime. A trustworthy measurement took two environments and two series: Docker/WSL2 drifted by more than 200 kRPS with the allocator held constant, and the first bare-metal series recorded no CPU time.

The corrected series reproduces the paper's release-on result. Across four balanced pairs the Temeraire delta is +0.401% in raw throughput and +0.474% in requests per CPU second. All four pairs are positive, and both 95% intervals exclude zero while containing the paper's +0.44%. Release-off centres on zero and stays unresolved across two independent samples. Normalization is worth under a tenth of a percentage point here, because a single-threaded Redis under a saturating benchmark holds one core throughout. The release rate the paper omits matters more: at 64 MiB/s every pair favors the legacy pageheap. The artifact, the per-trial data behind every number, and the source of this report are public at `https://github.com/Eyecatch3r/a-hugepage-aware-memory-allocator`.

1 Introduction

Memory allocation is often treated as a local implementation detail of C and C++ programs. Temeraire starts from a different observation: allocator decisions shape where objects land in memory, whether huge pages remain usable, and how much address-translation work the processor must perform later. Hunter et al. therefore connect allocator design to fleet efficiency as well as the direct cost of `malloc` and `free` [1].

A full reproduction of the Temeraire paper is outside the scope of this seminar artifact. The paper's main evidence comes from Google's production environment, including internal workloads, large-scale telemetry, and fleet-level rollout data. Those conditions are not available externally. This report therefore focuses on the Redis 6.0.9 case study: a public workload where the paper gives enough benchmark detail to support a local reproduction attempt.

The question here is narrower than rebuilding Google's internal allocator binary: can the Redis comparison be approximated with public code closely enough to observe the same small-effect regime? Reaching that point took two environments, a Docker/WSL2 stage for development and diagnosis, and a dedicated Linux node for the final native measurements.

The measured scope is two bare-metal series on node85. An earlier series covered three release rates with four order-alternated pairs each. A corrected series then ran 90.7 hours over eight balanced pairs and both readings of the paper's workload. Every allocator block holds 2000 trials, and an audit script re-derives every reported mean from the raw per-trial files. A Docker/WSL2 stage came first and is kept as a record rather than as evidence.

Artifact availability. The artifact is public at `https://github.com/Eyecatch3r/a-hugepage-aware-memory-allocator`. The repository holds the setup and benchmark scripts, the per-trial data of every run, the audit and aggregation tools, the unit tests, a static results explorer, and the source of this document. A clone regenerates every reported number and the one generated figure without a benchmark run. `python3 reproduce.py verify` performs both checks in seconds, and `README.md` documents the rest.

2 Hugepages, Fragmentation, and the TCMalloc Backend

Processors translate virtual addresses to physical addresses through page tables. Because page-table walks are expensive, recent translations are cached in translation lookaside buffers (TLBs). With ordinary 4 KiB pages, large working sets can require many translations. A 2 MiB hugepage covers the same address range as 512 ordinary pages, so one hugepage TLB entry can cover much more memory than one ordinary-page entry.

The gain comes from needing fewer cached translations, not from loading 2 MiB of data into cache at once. Whether that gain is realized depends on layout: hugepage mappings require suitably aligned and contiguous regions. Linux Transparent Huge Pages (THP) can create such mappings, but its success depends partly on how the user-space allocator arranges live objects.

Figure 1 shows the central trade-off. Returning small free pieces to the OS can reduce resident memory, but it may split hugepage mappings. Keeping the hugepage intact preserves TLB coverage, but leaves some physical memory idle. Temeraire exists to make this trade-off explicit inside the allocator backend.

TCMalloc is organized as a hierarchy of caches. Many small allocations are served by upper per-CPU, transfer, or central-cache layers. Larger page-granularity requests eventually reach the pageheap, which obtains memory from the OS and releases unused spans. Temeraire changes this backend pageheap layer and leaves the rest of the allocator alone.

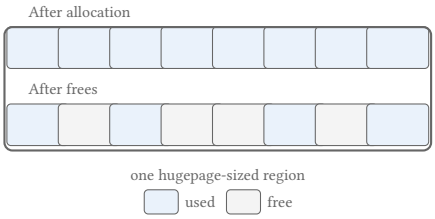

Figure 1: Fragmentation inside a hugepage-sized region. Free memory exists, but live objects prevent the whole region from being released intact. Redrawn after Figure 1 in Hunter et al. [1].

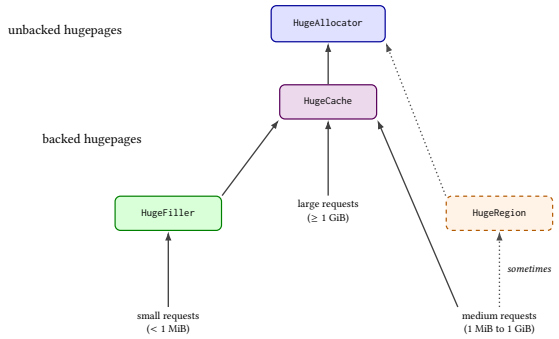

Figure 2: Temeraire's component structure. Small requests are routed to `HugeFiller`, large requests to `HugeCache`, and medium requests usually to `HugeCache` but occasionally to `HugeRegion`. Adapted after Figure 6 in Hunter et al. [1].

The experiment therefore compares two backend configurations of one allocator, the legacy pageheap and the hugepage-aware path, rather than two separate allocators.

3 How Temeraire Places Memory

Figure 2 summarizes the backend components relevant for the artifact. `HugeAllocator` manages hugepage-aligned virtual address ranges. `HugeCache` keeps completely empty backed hugepages for reuse. `HugeFiller` handles small allocations and is the key component for Redis. `HugeRegion` handles medium-sized allocations that would otherwise create large slack regions.

The key Redis-relevant policy is in `HugeFiller`. The paper describes it, and the pinned revision that the artifact builds carries the detail in source comments, so this section follows the source. `HugeFiller` holds every partly used hugepage in a list and indexes those lists by two properties.

The first property is the longest free run inside the hugepage. It is a proxy for fragmentation, because a short longest run means the free space is scattered. It also answers the allocation question directly, since a hugepage fits a request of K pages only if its longest free run reaches K. For such a request, `HugeFiller` takes the hugepage whose longest free run is smallest among those that still fit, which keeps the long runs for the requests that need them.

The second property is the number of live allocations, quantized into eight logarithmic chunks. Ties on the first property break toward the hugepage that already holds more allocations. The source counts allocations rather than allocated pages, because allocations die at roughly the same rate whatever their size: a hugepage with two five-page allocations empties sooner than one with five single-page allocations, though a count of pages ranks the two the same.

A third rule exists because the first two once caused production fragmentation. When a large request leaves a tail, that tail is donated to `HugeFiller`, which then prefers regular hugepages over freshly donated ones whatever their free-run measure, so that a donated tail can rejoin its large range once that range is freed.

The Redis list workload in the paper performs small list-node and string-value allocations. These do not reach `HugeFiller` individually. The per-CPU and central caches serve them, and only when those caches need fresh backing spans does a page-level request arrive at the pageheap. `HugeFiller` places those spans, and the legacy pageheap places them without a hugepage-aware objective.

Periodic memory release complicates the picture. When release is enabled, TCMalloc can return memory to the OS using mechanisms such as `MADV_DONTNEED`. The pinned revision adds one policy that matters for Section 6.4: once part of a hugepage goes back to the OS, the allocator returns anything else that falls free inside it. A higher release rate therefore breaks more hugepages, and a broken hugepage keeps giving memory back. Release interacts with hugepage coverage and adds variance, so release-off carries the cleaner placement signal of the two conditions.

4 Reproduction Target and Artifact Evolution

The paper gives Redis 6.0.9, a legacy TCMalloc pageheap baseline, 2000 `redis-benchmark` trials, 1,000,000 requests per trial, and a workload combining a five-element push with a five-element read [1]. It also states the CPU family and LLVM commit used for compilation. The exact Linux distribution, kernel version, and THP policy for the Redis run are left unstated, so this report documents them locally instead of matching them.

The repository did not initially match this target. Its defaults used Redis 7.2.5 and compared glibc against open-source gperftools TCMalloc, with no clean legacy-pageheap versus Temeraire split. The first step was to narrow the claim to the Redis case-study methodology alone.

The paper's internal allocator build is unavailable, so the artifact uses a historical public `google/tcmalloc` commit, `8e534f50 7074`, which still exposes the `want_no_hpaa` mechanism needed to disable the hugepage-aware path. This makes a public legacy-versus-Temeraire split possible, but it should not be read as byte-identical to Google's internal artifact.

The public repository provides no ready-made shared libraries for Redis preloading. The setup script adds a small wrapper target inside the cloned TCMalloc tree and builds two loadable variants. The Temeraire build links against `//tcmalloc`, and the legacy build also links against `//tcmalloc:want_no_hpaa`. The harness selects the library through `LD_PRELOAD`.

Bazel let the artifact use TCMalloc's native build graph instead of reconstructing allocator dependencies, compiler flags, and link semantics elsewhere. It also kept the comparison narrow: both

libraries come from the same wrapper source, and the intended difference is the allocator target plus the extra `//tcmalloc:want_no_hpaa` dependency. The cost was integration friction: a pinned Bazelisk, the LLVM path threaded through Bazel's action and repository environments, an external workspace that inherited no Abseil dependency, and target visibility that forced the wrapper into `tcmalloc/testing`.

Runtime verification is necessary. Redis reports `mem_allocator:libc` from its own build metadata even when `LD_PRELOAD` has replaced the process allocator, so the artifact checks `/proc/$pid/maps` to confirm that the intended shared library is mapped into the Redis process.

A later validation run uncovered a symbol mismatch: the wrapper referenced background-release methods absent from the pinned revision, so the shared objects built but failed to load. The fix resolves those methods dynamically when available.

Compiler matching cost more than it looked. The paper mentions LLVM commit `cd442157cf` and `-O3`, so the artifact built a local LLVM 13.0.0 toolchain from that commit and used it for Redis, gperftools, and the TCMalloc wrapper. One older manifest still marks the exact LLVM path as disabled, so the recorded compiler metadata is treated as authoritative.

The most important methodological difficulty was THP. Initial long runs under WSL2/Docker with THP in `madvise` mode recorded `AnonHugePages: 0 kB` for Redis, so they looked like controlled allocator comparisons while leaving the hugepage mechanism unevidenced. The harness now records THP settings and periodic `smaps_rollup` snapshots, and the paper-closer runs set THP to `always` before benchmarking. Even then, later release-on runs showed large changes in absolute host throughput.

5 Methodology

5.1 Common benchmark path

The repository retains a short direct Redis path for smoke tests and allocator preload checks. Measurements reported here use the paper-closer path. It fixes Redis to version 6.0.9 and uses 2000 trials, 50 clients, and pipeline depth 16. Each trial flushes the database and then runs two separate `redis-benchmark` invocations of 1,000,000 requests each:

- `LPUSH benchmark:list v1 v2 v3 v4 v5`;
- `LRANGE benchmark:list 0 4`.

This structure is one reading of an underspecified sentence. The paper specifies 2000 trials, each making one million requests "to push 5 elements and read those 5 elements" [1]. Read literally, one request covers both operations, so a trial performs one million of each. The local trial instead issues one million pushes and then one million reads, which doubles the command count and separates the phases: because all pushes precede all reads, `LRANGE 0 4` returns the same final five elements on every read while the list holds five million. An interleaved implementation would keep the live set small and touch freshly written memory, which is a different live-heap shape for the allocator under test. Section 6 measures both readings.

The runner stores the raw output of every operation as well as summary files, Redis logs, memory snapshots, allocator order, software revisions, and system metadata. Allocator order alternates between pairs, which does not remove drift between consecutive blocks but avoids giving the same allocator the later position every time.

The primary metric is throughput in requests per second from `redis-benchmark`. LPUSH and LRANGE are collapsed into a combined push-plus-read rate using the harmonic mean

$$\frac{2}{1/r_{\mathrm{LPUSH}} + 1/r_{\mathrm{LRANGE}}}.$$

This treats the two throughputs as rates for sequential operations and reduces each block to one number, as the paper's single Redis delta per release mode does. The harmonic mean is a local reconstruction choice: the public `redis-benchmark` harness records LPUSH and LRANGE as separate operation means, while the paper reports one delta for the combined workload.

This metric is not the paper's metric, and the difference bounds every comparison in Section 6. The caption of Table 1 in Hunter et al. states that its throughput is "normalized for CPU", and the surrounding text gives the unit as requests per second per core [1]. The local figure is the raw request rate. A ratio of unnormalized rates cannot separate a real throughput gain from the same throughput bought with more CPU, which is the distinction the paper's fleet-efficiency argument rests on. Where this report places a local value near a published one, that is proximity between two differently constructed quantities rather than agreement between two instances of one measurement.

The hypothesis is modest, and follows from Section 3. If the placement heuristic in `HugeFiller` improves hugepage coverage for the spans backing Redis's small allocations, Redis shows a small positive throughput delta against the legacy pageheap. The effect is clearest with periodic release disabled, since release-on adds interaction with background memory return and can mask the placement signal.

5.2 Docker/WSL2 development stage

The first complete setup used a Debian 12 container on Docker Desktop and its shared WSL2 kernel. It carried the pinned builds, preload checks, benchmark parameters, THP instrumentation, and balanced ordering. Once THP was set to `always`, Redis reported non-zero hugepage use. The container stayed a good development environment and made a poor measuring instrument, for reasons Section 6.5 gives.

Two faults found in that stage shaped everything after it. The runner started TCMalloc's background-actions thread without setting a release rate, which the pinned revision defaults to zero, so nothing labelled release-on had performed periodic release. The label had been wrong for the whole series. Fixing it exposed a second fault: the wrapper's dynamic symbol lookups had been failing silently, and `LD_PRELOAD` had leaked from the Redis server to `redis-cli` and `redis-benchmark`, mixing client and server allocators into the same comparison. The wrapper now calls the linked TCMalloc API directly, the preload is scoped to the server, and every release-on run must log its requested rate before the benchmark proceeds.

Neither fix was the end of it. Rebuilding the wrapper needed one further Bazel dependency, and the container scripts still carried Windows line endings, which Linux reports as a missing `bash\r` interpreter rather than as a line-ending problem. Both cost more

time than the allocator change they were meant to enable, which is the ordinary shape of this work: the substantive edit is small and the environment around it is not.

5.3 Bare-metal Linux stage

The WSL2 drift motivated a second workflow rather than a change to the existing container scripts. The native scripts preserve the same pinned source revisions, allocator wrappers, Redis workload, result layout, and balanced pair structure. They run directly on node85, a machine with an Intel Xeon Gold 5318N (24 cores, 48 threads, one NUMA node) and 188 GiB of memory, running Debian GNU/Linux 13 (trixie) on kernel `6.12.95+deb13-amd64` with no virtualization detected. THP is set to `always` with defrag `madvise`.

The move exposed a different set of practical problems. The shared home directory was too slow for Bazel's many small files, so the build and runs moved to `/var/tmp`. The compute node could not fetch the pinned sources, so the build ran offline from staged archives. The old LLVM revision needed `-include cstdint` under Debian 13's host compiler, and the German locale produced decimal commas in CSV files, so the native runner now sets `LC_ALL=C`.

After three smoke runs and two ten-trial pilots, four full balanced pairs ran at 16 MiB/s, each pair covering legacy and Temeraire with release disabled and enabled. A second series kept release enabled and added four order-alternated pairs at 64 MiB/s and four at 256 MiB/s. All full allocator blocks contain 2000 trials, and each release-on Redis log confirms the requested rate. The archive holds 64,000 trial rows for the main four pairs and another 64,000 for the two added rates. A standard-library audit script re-derives every block mean from the raw trial files, checks trial identifiers, request counts, file counts, memory samples, THP state, release-rate confirmations, clean shutdowns, allocator order, and system metadata, and fails on any mismatch. Both halves of the archive pass.

6 Results

6.1 Three faults that forced a second series

The first bare-metal series was complete and audited, and it was measuring the wrong thing. Three problems surfaced in an order worth recording, because the last two were faults in the harness that produced those very numbers.

The first came from reading the paper again with the harness open alongside it. The metric gap of Section 5 was worse than a caveat here, because the artifact recorded no CPU time at all. The same reading showed that the paper's trial sentence supports a second implementation, and that its Redis servers used Skylake Xeons while node85 carries an Ice Lake part. Two of those three gaps could be closed by changing the harness. The processor could not.

The second problem appeared eighteen hours in. A block died at trial 1875 of 2000, and its Redis log showed a clean shutdown, which made the failure look transient. It was not. The run sat inside an SSH session that ended. Redis ignores `SIGHUP` at startup and `redis-benchmark` does not, so the hangup killed the client and the exit trap then closed the server. The clean shutdown was the consequence of the failure rather than evidence against it.

The third problem destroyed a day of compute. `redis-benchmark` builds its CSV test name from the command line, so an `EVAL` run carries the whole Lua source in that field, commas included. The harness split the line on commas and took the second field, which for such a run is the fragment `KEYS[1]`, so a sixteen-hour block came out with a mean of zero. Plain `LPUSH` and `LRANGE` names carry no commas, which is why the fault stayed hidden until the second workload ran. It was recoverable, because the runner writes each invocation's output to its own file first, and a separate utility re-derives `trials.csv` from those files. The episode still exposes a weakness in the audit: the summary and the trial table descend from the same extraction, so it caught this only because `KEYS[1]` is not a number. A plausible but wrong value would have passed.

This section records only the faults that changed a result or a method. The artifact carries a longer protocol note listing every dead end in order, including those that cost time without changing a number.

Four changes followed. Each block now records the CPU time of the Redis server from `redis-cli info cpu`, which is that block's total because Redis restarts for every block. The workload is selectable, so both readings of the paper's trial sentence can be measured. A failed invocation is retried, and every attempt reaches the audit as a warning. Runs start in their own session through `setsid`.

6.2 Reported result

Every block of the corrected series holds 2000 trials, every release-on log confirms the 16 MiB/s rate, no block needed a retry, and both launcher status files record a zero exit code. Of the 32 blocks, 28 re-derive byte-identical summaries from their raw per-trial files. The four that differ are the set damaged by the extraction fault, repaired as described above.

Table 1: Reported result. Correction run on node85, sequential workload. Deltas in raw throughput and in requests per CPU second, the unit the paper reports. Intervals are four-pair Student-t intervals on the log-transformed pair ratios and accompany the mean.

Pair	Order	Release off	Release on
5	L first	−0.476%	+0.242%
6	T first	+0.156%	+0.555%
7	L first	−0.083%	+0.380%
8	T first	+0.490%	+0.426%
Mean		+0.022%	+0.401%
Median		+0.037%	+0.403%
95% interval		[−0.62, +0.67]	[+0.20, +0.61]
Mean per CPU second		+0.091%	+0.474%
95% interval		[−0.74, +0.93]	[+0.30, +0.65]
Paper		+0.75%	+0.44%

Release-on at 16 MiB/s reproduces the paper. All four pairs favor Temeraire and the mean is +0.401%, with an interval that excludes zero. Normalizing for CPU moves the mean to +0.474%, and that interval excludes zero as well. The paper's +0.44% falls inside both. It is the only condition in the study that separates from the baseline in Temeraire's favor. Absolute throughput held between 980 and 995 kRPS across all sixteen blocks, a spread of 1.6%.

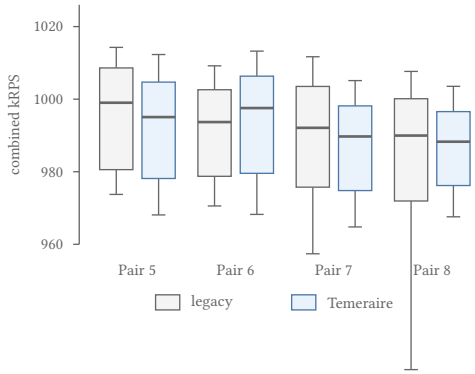

Figure 3: Per-trial combined throughput for the four release-off pairs of the correction run. Boxes span the interquartile range, whiskers the 5th to 95th percentile, and the heavy line the median of 2000 trials. The within-block spread is roughly 4%, an order of magnitude wider than the pairwise deltas in Table 1.

Release-off does not resolve. The mean is +0.022% and the median +0.037%, with an interval that covers zero and, once normalized, covers the paper's +0.75% as well. Figure 3 shows why four pairs are not enough here: the per-trial distributions inside each block span roughly 4%, an order of magnitude wider than the pairwise deltas that separate them, so the allocator difference is a small shift between two heavily overlapping populations.

Normalizing for CPU changes little, and the CPU record explains why. Sequential blocks spent about 4040 CPU seconds across roughly 4100 seconds of wall clock, and combined blocks 14 860 across 14 900, so the Redis process held between 98 and 99.7 percent of a single core throughout. Redis 6.0.9 executes commands on one thread and the benchmark keeps that thread saturated, so requests per second and requests per second per core are nearly the same quantity here. The metric gap of Section 5 is therefore real but worth under a tenth of a percentage point. That conclusion is specific to a single-threaded saturated server and does not carry over to the multithreaded applications in the paper's Table 1.

6.3 The second reading of the workload

Table 2: Correction run on node85, combined `EVAL` workload, where one request performs the push and the read. Same interval construction as Table 1.

Pair	Order	Release off	Release on
1	L first	−0.260%	−1.121%
2	T first	+0.817%	−0.085%
3	L first	+1.287%	−0.304%
4	T first	+0.867%	+2.506%
Mean		+0.678%	+0.249%
Median		+0.842%	−0.194%
95% interval		[−0.37, +1.73]	[−2.21, +2.75]

The combined reading costs far more and measures far less. A block takes 4.11 hours against 1.14, because an `EVAL` request carries Lua interpreter work, and its throughput sits near 135 kRPS rather than 990. Its release-off median of +0.842% lands near the paper's +0.75%, but with an interval more than two percentage points wide that agreement carries little weight. Its release-on column is worse still: one pair at +2.506% drives a positive mean out of three negative pairs, and the interval spans five percentage points. CPU normalization shifts these figures by less than 0.01 percentage points, because these blocks are the most saturated of the study.

Running both readings was worth the compute, because it shows that the choice the paper's sentence leaves open is not a detail. The sequential form is the better instrument, and Table 1 rests on it.

6.4 Release-rate sensitivity

The 16 MiB/s rate is a local parameter, since the paper never states one. The correction run measured only that rate, so the evidence on higher rates comes from the earlier series of July 2026, which used the same node, workload, and pair structure but recorded no CPU time. Its figures are therefore unnormalized, and Section 6.2 shows that for this workload the normalization would move them by under a tenth of a percentage point.

Table 3: Release-on sensitivity from the earlier series. Each cell compares Temeraire with the matched legacy block, and the four pairs alternate allocator order. Unnormalized throughput.

Rate	Pair 1	Pair 2	Pair 3	Pair 4	Mean	Median
16 MiB/s	+1.319%	+0.149%	−0.358%	+0.942%	+0.513%	+0.545%
64 MiB/s	−0.631%	−0.094%	−0.674%	−0.574%	−0.494%	−0.603%
256 MiB/s	−0.028%	+0.402%	−0.133%	−1.419%	−0.294%	−0.080%

All four 64 MiB/s pairs favor legacy TCMalloc, and their interval, [−0.92%, −0.07%], excludes zero. It is the only interval here that does so, which is worth recording but is weak evidence on its own: it rests on four pairs, on a normal approximation, and on one condition picked out of several tested without any correction for multiple comparisons. At 256 MiB/s three pairs are negative and one is positive; the −1.419% pair drives the negative mean, and inspection by trial quarter rules out a short collapse in that block, with Temeraire between 1.2% and 1.8% slower in each quarter. The sequence is not monotonic: performance changes sign between 16 and 64 MiB/s, then moves back toward zero at 256 MiB/s. The rerelease policy of Section 3 is one candidate mechanism, since a higher rate breaks more hugepages and each broken hugepage keeps returning memory. Four pairs cannot test that explanation, and this report does not claim it. Whatever the release rate does to this workload, it does not do it smoothly.

6.5 What the earlier series established

Two findings from the earlier work still carry weight.

The Docker/WSL2 stage could not carry a sub-one-percent claim. Absolute throughput moved by more than 200 kRPS across a single series and later recovered. Each allocator block ran for about an hour, so a block inherited whatever the host happened to be doing, and the largest apparent Temeraire improvements fell in the blocks that ran while the host recovered from a slower period. That correlation is what makes the numbers uninterpretable rather than

merely noisy: the allocator delta cannot be separated from the hour in which each block ran. Its release-on median reached +6.80% at 64 MiB/s, one 256 MiB/s pair reached +15.73%, and one release-on pair reached −12.02% and did not reproduce when rerun. All three are an order of magnitude larger than the effect under study. The release-rate fault of Section 5 covers this whole stage, so its logs are retained as a record of the misconfiguration and excluded from every comparison.

The first bare-metal series, run in July on the same node, is a second sample of release-off. It gave a median of −0.34% against the correction run's +0.037%, with intervals that overlap across almost their whole width. Two four-pair samples that disagree in sign while agreeing within their intervals call for more pairs. Its release-on pairs point the same way, so seven of the eight release-on pairs on node85 favor Temeraire.

7 Discussion and Related Work

The move to bare metal narrows one question raised by WSL2. The double-digit release-on deltas did not recur on node85, where every pair stays within 1.5%, so they were environment- or time-dependent rather than a property of Temeraire. The rerun isolates no cause, because it changed CPU generation, kernel, operating system, and time period together.

Reproducing the paper's release-on value is not the same as confirming the paper's claim, and the difference is the release rate. Since the paper omits its rate, 16 MiB/s has no claim to being the setting behind +0.44%, and the 64 MiB/s aggregate runs the other way with an interval that also excludes zero. An effect that reverses between two rates the paper never distinguishes is weaker than a +0.44% headline suggests, and Table 3 is the part of this study to weigh against it.

Two measurement gaps limited how far these comparisons could be pushed, and Section 6.2 measured both. The metric gap is worth under a tenth of a percentage point for this workload, because a single-threaded Redis under a saturating benchmark makes the two units almost the same quantity. The workload gap is larger: the two readings of the paper's trial sentence differ by a factor of 3.6 in cost and by an order of magnitude in interval width. Neither gap is closed by argument, but both are now quantified.

Four pairs per condition remain few for sub-one-percent effects. Allocator blocks run consecutively, so slow changes in frequency, temperature, or system activity survive inside a pair. The metadata omits CPU governor, turbo state, and per-block allocator mappings, and preload was verified once before the run rather than inside every result directory.

Environmental fidelity also remains limited. The paper leaves the Linux distribution, kernel, THP policy, and background release rate unstated, and one property it does specify was not matched: its Redis servers used Intel Skylake Xeons, while node85 carries an Ice Lake part. TLB capacity and hugepage handling changed between those generations, so the hardware most relevant to a TLB-pressure result is the piece of the paper's stated environment that this reproduction does not reproduce.

Temeraire sits alongside other work on huge pages. SuperMalloc uses them for very large allocations but leaves the general pageheap layout hugepage-unaware [3]. MallocPool improves TLB behavior by serving objects from a contiguous preallocated region, but it does not replace the pageheap [2]. LLAMA groups allocations by predicted lifetime, whereas Temeraire places online without profiling [5]. Kernel-level systems such as Ingens and HawkEye manage huge pages from the OS side, complementary to allocator-side placement [4, 6].

8 Conclusion

This report reproduces the Redis case study alone, on an artifact narrowed from a generic allocator benchmark to Redis 6.0.9 and from glibc against gperftools to legacy TCMalloc against Temeraire. WSL2 host drift made the first stage unusable as final evidence, and bare-metal Linux removed those swings.

Release-on at 16 MiB/s reproduces the paper. Across four fresh pairs Temeraire gains +0.401% in raw throughput and +0.474% in requests per CPU second, every pair is positive, and both intervals exclude zero while containing the paper's +0.44%. Seven of the eight release-on pairs measured on node85 favor Temeraire. Release-off stays unresolved: two independent four-pair samples put it at −0.34% and +0.037% with intervals that overlap almost entirely, so the earlier reading of the negative sample as inconsistent with the paper does not survive the second.

Two of the three known deviations are now measured rather than assumed: CPU normalization is worth under a tenth of a percentage point for a single-threaded saturated Redis, and the two readings of the paper's trial sentence differ by a factor of 3.6 in cost and an order of magnitude in interval width. The third, Skylake against Ice Lake, needs different hardware. Beyond the numbers, the reproduction adds an audited path through a misconfigured release rate, a signal that killed a run, and a parsing fault that silently zeroed a day of measurements. Google's fleet-scale claims stay outside its reach.

References

[1] A.H. Hunter, Chris Kennelly, Paul Turner, Darryl Gove, Tipp Moseley, and Parthasarathy Ranganathan. 2021. Beyond malloc efficiency to fleet efficiency: a hugepage-aware memory allocator. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. USENIX Association, 257–273. https://www.usenix.org/conference/osdi21/presentation/hunter

[2] Marcus Jägemar. 2018. Mallocpool: Improving Memory Performance Through Contiguously TLB Mapped Memory. In *2018 IEEE 23rd International Conference on Emerging Technologies and Factory Automation (ETFA)*, Vol. 1. 1127–1130. doi:10.1109/ETFA.2018.8502619

[3] Bradley C. Kuszmaul. 2015. SuperMalloc: a super fast multithreaded malloc for 64-bit machines. In *Proceedings of the 2015 International Symposium on Memory Management* (Portland, OR, USA) *(ISMM '15)*. Association for Computing Machinery, New York, NY, USA, 41–55. doi:10.1145/2754169.2754178

[4] Youngjin Kwon, Hangchen Yu, Simon Peter, Christopher J. Rossbach, and Emmett Witchel. 2016. Coordinated and efficient huge page management with ingens. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation* (Savannah, GA, USA) *(OSDI'16)*. USENIX Association, USA, 705–721.

[5] Martin Maas, David G. Andersen, Michael Isard, Mohammad Mahdi Javanmard, Kathryn S. McKinley, and Colin Raffel. 2020. Learning-based Memory Allocation for C++ Server Workloads. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) *(ASPLOS '20)*. Association for Computing Machinery, New York, NY, USA, 541–556. doi:10.1145/3373376.3378525

[6] Ashish Panwar, Sorav Bansal, and K. Gopinath. 2019. HawkEye: Efficient Fine-grained OS Support for Huge Pages. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) *(ASPLOS '19)*. Association for Computing Machinery, New York, NY, USA, 347–360. doi:10.1145/3297858.3304064